GSFC UNIVERSITY

Course: Generative AI and Deep Learning

UNIT 1

Foundations of Generative AI

Detailed Study Notes

Overview of AI, ML & Deep Learning | Introduction to Generative Models | Probability Distributions | Types of Generative Models (GANs, VAEs, Diffusion) | Key Applications of Generative AI

Prepared for classroom and self-study use. Covers every syllabus sub-topic with definitions, formulas, worked explanations, comparison tables and exam-style key points.

Table of Contents

1. Overview of Artificial Intelligence, Machine Learning, and Deep Learning
    1.1 Artificial Intelligence — Definition, History, Types
    1.2 Machine Learning — Definition, Types, Workflow
    1.3 Deep Learning — Neural Networks, Why Deep Learning Works
    1.4 AI vs ML vs DL — Relationship and Comparison
2. Introduction to Generative Models: Concepts and Scope
    2.1 What Is a Generative Model?
    2.2 Generative vs Discriminative Models
    2.3 Scope, History and Why Generative AI Matters Now
3. Probability Distributions: Gaussian, Multivariate, Sampling
    3.1 Probability Distribution Basics
    3.2 The Gaussian (Normal) Distribution
    3.3 The Multivariate Gaussian Distribution
    3.4 Sampling Methods
4. Types of Generative Models: GANs, VAEs, Diffusion Models
    4.1 Generative Adversarial Networks (GANs)
    4.2 Variational Autoencoders (VAEs)
    4.3 Diffusion Models
    4.4 Comparative Summary Table
5. Key Applications of Generative AI
    5.1 Image Synthesis
    5.2 Text Generation
    5.3 Drug Discovery
    5.4 Other Applications (Audio, Video, Code, Data Augmentation, etc.)
Unit 1 — Quick Revision Summary

1. Overview of Artificial Intelligence, Machine Learning, and Deep Learning

1.1 Artificial Intelligence (AI)

DEFINITION — Artificial Intelligence

The branch of computer science concerned with building systems (machines/software) that can perform tasks which normally require human intelligence — such as reasoning, learning, perception, planning, language understanding, and decision-making.

Historical milestones students should remember:

- **1950** — Alan Turing proposes the 'Turing Test' in his paper *Computing Machinery and Intelligence* to judge machine intelligence.
- **1956** — Dartmouth Conference: John McCarthy coins the term **'Artificial Intelligence'**; considered the birth of AI as a field.
- **1960s–70s** — Early symbolic AI / rule-based expert systems (e.g., ELIZA, MYCIN).
- **1980s** — Rise of expert systems in industry; first 'AI Winter' follows due to unmet expectations and funding cuts.
- **1990s** — Machine learning approaches (statistical learning) gain popularity; IBM's Deep Blue beats chess champion Garry Kasparov (1997).
- **2006 onward** — 'Deep Learning' era begins with Geoffrey Hinton's work on deep belief networks.
- **2012** — AlexNet wins ImageNet competition, proving deep CNNs outperform traditional methods — sparks the modern deep learning boom.
- **2014** — Generative Adversarial Networks (GANs) introduced by Ian Goodfellow.
- **2017** — Transformer architecture ('Attention Is All You Need') published — foundation of modern LLMs.
- **2020s** — Explosion of Generative AI: GPT series, DALL·E, Stable Diffusion, ChatGPT — mainstream adoption.

Types / Categories of AI (students must know both classification schemes):

(a) Based on Capability:

- **Narrow AI (Weak AI):** Designed and trained for a specific task (e.g., voice assistants, spam filters, recommendation engines). All current real-world AI is narrow AI.
- **General AI (Strong AI / AGI):** Hypothetical AI with human-level cognitive ability across any intellectual task. Does not yet exist.
- **Super AI (ASI):** A theoretical future stage where machine intelligence surpasses human intelligence in all respects.

(b) Based on Functionality:

- **Reactive Machines:** No memory, respond only to current input (e.g., IBM's Deep Blue).
- **Limited Memory:** Uses past experience/data briefly to make decisions (e.g., self-driving cars).
- **Theory of Mind:** (Research stage) Would understand emotions, beliefs, intentions of others.
- **Self-Aware AI:** (Hypothetical) Possesses consciousness and self-awareness.

1.2 Machine Learning (ML)

DEFINITION — Machine Learning

A subset of AI in which systems learn patterns and rules automatically from data, and improve their performance on a task through experience, without being explicitly programmed with fixed rules. Arthur Samuel (1959) defined it as the 'field of study that gives computers the ability to learn without being explicitly programmed.'

Core Types of Machine Learning:

Type	Description	Examples
Supervised Learning	Model learns a mapping from labelled input–output pairs (X → y). Includes classification (discrete output) and regression (continuous output).	Spam detection, house price prediction, image classification
Unsupervised Learning	Model finds hidden structure/patterns in unlabelled data. Includes clustering, dimensionality reduction, and — importantly for this course — density estimation / generative modelling.	Customer segmentation (K-Means), PCA, GANs, VAEs
Semi-Supervised Learning	Uses a small amount of labelled data with a large amount of unlabelled data during training.	Web content classification, medical imaging with few labels
Reinforcement Learning	An agent learns to take actions in an environment to maximize cumulative reward through trial and error, using a reward signal.	Game playing (AlphaGo), robotics, RLHF for LLMs

Typical ML Workflow (important sequence to memorize):

- **1. Data Collection** — gathering raw data relevant to the problem.
- **2. Data Pre-processing** — cleaning, normalization, handling missing values, feature engineering.
- **3. Splitting Data** — into training, validation, and test sets.
- **4. Model Selection** — choosing an appropriate algorithm/architecture.
- **5. Training** — fitting model parameters to minimize a loss function using an optimizer (e.g., gradient descent).
- **6. Evaluation** — measuring performance with metrics (accuracy, precision/recall, F1, RMSE, etc.).
- **7. Hyperparameter Tuning** — adjusting learning rate, batch size, regularization, etc.
- **8. Deployment & Monitoring** — putting the model into production and tracking performance/drift.

1.3 Deep Learning (DL)

DEFINITION — Deep Learning

A subfield of machine learning based on **artificial neural networks with multiple (deep) layers** that automatically learn hierarchical feature representations directly from raw data (pixels, audio waveforms, text tokens), reducing the need for manual feature engineering.

Why Deep Learning became dominant — key reasons students should list in exams:

- **Big Data availability** — internet-scale datasets (ImageNet, Common Crawl) provide enough examples to train large models.
- **Compute power** — GPUs/TPUs enable massively parallel matrix computation needed for training deep nets.
- **Algorithmic advances** — ReLU activation, dropout, batch normalization, better optimizers (Adam), residual connections (ResNet), and attention mechanisms (Transformers).
- **Automatic feature learning** — unlike classical ML which needs hand-crafted features, deep nets learn low-level to high-level features layer by layer (edges → shapes → objects, for example, in a CNN).
- **End-to-end learning** — a single network can be trained directly from raw input to final output.

Building blocks of a neural network (foundational vocabulary):

- **Neuron / Perceptron:** Computes a weighted sum of inputs plus a bias, passed through an activation function.
- **Layers:** Input layer, one or more hidden layers, and an output layer.
- **Weights & Biases:** Learnable parameters adjusted during training.
- **Activation Functions:** Introduce non-linearity — Sigmoid, Tanh, ReLU, Leaky ReLU, Softmax (for output).
- **Forward Propagation:** Passing input through the network to get an output/prediction.
- **Loss Function:** Measures the difference between predicted and actual output (e.g., MSE, Cross-Entropy).
- **Backpropagation:** Algorithm that computes gradients of the loss with respect to each weight using the chain rule.
- **Gradient Descent / Optimizers:** Update weights in the direction that reduces the loss (SGD, Adam, RMSProp).
- **Common Architectures:** CNNs (images), RNNs/LSTMs (sequences), Transformers (sequences, attention-based) — these become the backbone of modern generative models.

1.4 AI vs ML vs DL — The Relationship

The three fields are **nested subsets, not separate parallel fields**: **AI ⊃ Machine Learning ⊃ Deep Learning**. In other words, all Deep Learning is Machine Learning, and all Machine Learning is Artificial Intelligence, but the reverse is not true — e.g., a rule-based chess engine is AI but not ML.

Aspect	Artificial Intelligence	Machine Learning	Deep Learning
Definition	Broad field: machines mimicking human intelligence	Subset of AI: learning from data	Subset of ML: learning via multi-layer neural networks
Data dependency	Varies (rule-based systems need little data)	Needs moderate–large labelled/unlabelled data	Needs very large datasets to perform well
Feature engineering	N/A / manual	Mostly manual (hand-crafted features)	Automatic (learned hierarchically)
Hardware need	Low (for symbolic AI)	Moderate	High (GPU/TPU essential)
Interpretability	High (rule-based)	Moderate	Low ('black box')

Aspect	Artificial Intelligence	Machine Learning	Deep Learning
Examples	Chess engines, expert systems, chatbots	Spam filters, decision trees, SVMs	CNNs, RNNs, Transformers, GANs, VAEs

★ Exam Key Points — Section 1

• AI is the broadest goal; ML is a data-driven approach to achieve AI; DL is a specific technique within ML using deep neural networks.

• Turing Test (1950) and Dartmouth Conference (1956) are the two most-asked historical facts.

• Supervised, Unsupervised, Semi-supervised and Reinforcement Learning are the 4 core ML paradigms — Generative Models mainly fall under Unsupervised Learning (density estimation).

• Deep Learning's rise (post-2012, AlexNet) was driven by data + compute + algorithms, not any single factor.

2. Introduction to Generative Models: Concepts and Scope

2.1 What Is a Generative Model?

DEFINITION — Generative Model

A class of machine learning models that learn the underlying **probability distribution** of a dataset, p(x) (or the joint distribution p(x, y)), so that they can **generate new, synthetic data samples** that resemble the training data. Instead of only predicting a label for a given input, a generative model can produce brand-new outputs — images, text, audio, molecules — that did not exist in the original dataset.

In simple terms: a generative model answers the question *'How could this kind of data have been produced?'* and then uses that learned understanding to create new examples. For instance, after seeing thousands of handwritten digit images, a generative model learns the general concept of what makes a digit '7' look like a '7' and can then draw new, never-before-seen '7' images.

2.2 Generative vs Discriminative Models

This is one of the most important conceptual distinctions in the whole unit and is frequently asked as a direct exam question.

Aspect	Discriminative Models	Generative Models
What they learn	The decision boundary / conditional probability P(y|x) — i.e., given input x, predict label y	The underlying data distribution P(x) or joint P(x, y) — how the data itself is distributed
Goal	Classify or predict directly	Model and generate new data samples
Can generate new data?	No	Yes
Examples	Logistic Regression, SVM, standard CNN classifiers, Discriminator network in a GAN	Naive Bayes, Gaussian Mixture Models, GANs, VAEs, Diffusion Models, autoregressive language models (GPT)
Analogy	Learns to tell real from fake / cat from dog	Learns what makes something look like a cat, and can draw a new cat

Mathematically: a discriminative model directly estimates P(y | x). A generative model estimates P(x) or P(x, y), and can use Bayes' theorem P(y|x) = P(x|y)P(y) / P(x) if classification is also needed. Because generative models capture the full data distribution, they require modelling more complexity than discriminative models, but in return they gain the ability to sample/create new data.

2.3 Scope of Generative AI and Why It Matters Now

Scope covers multiple data modalities — students should be able to name at least 4–5:

- **Images:** photorealistic image synthesis, art generation, super-resolution, inpainting.

- **Text:** story/article writing, summarization, translation, chatbots, code generation.
- **Audio/Speech:** voice cloning, music composition, text-to-speech.
- **Video:** video synthesis, deepfakes, frame interpolation.
- **3D content:** 3D object/scene generation for gaming, AR/VR, CAD design.
- **Scientific data:** molecule and protein structure generation for drug discovery and materials science.
- **Tabular / structured data:** synthetic data generation for privacy-preserving analytics and data augmentation.

Why Generative AI is significant today:

- Availability of massive unlabelled datasets suits unsupervised generative approaches.
- Advances in deep architectures (Transformers, U-Nets) and compute made high-fidelity generation feasible.
- Wide commercial impact: content creation, design, software engineering, healthcare, education.
- Foundation for modern conversational AI systems (large language models) and multimodal assistants.
- Enables data augmentation where real data is scarce, expensive, or privacy-sensitive.

★ Exam Key Points — Section 2

• Generative models learn P(x) or P(x,y); discriminative models learn P(y|x) directly.
• Only generative models can synthesize new, realistic samples.
• Generative modelling is fundamentally a form of unsupervised/self-supervised density estimation.
• Scope spans images, text, audio, video, 3D, and scientific/biological data.

3. Probability Distributions: Gaussian, Multivariate, and Sampling Methods

Generative models are fundamentally built on probability theory — they learn a distribution and then draw ('sample') from it. This section provides the mathematical foundation needed before studying GANs, VAEs, and Diffusion Models.

3.1 Probability Distribution — Basics

DEFINITION — Probability Distribution

A mathematical function that gives the probabilities of occurrence of different possible outcomes of a random variable. For a **discrete** random variable it is called a Probability Mass Function (PMF); for a **continuous** random variable it is called a Probability Density Function (PDF).

- **Random Variable:** A variable whose value is a numerical outcome of a random phenomenon.
- **PDF properties:** $f(x) \geq 0$ for all x, and the total area under the curve integrates to 1: $\int f(x)\,dx = 1$.
- **CDF (Cumulative Distribution Function):** $F(x) = P(X \leq x)$; it is the integral of the PDF.
- **Expectation (Mean, μ):** The average/expected value of the random variable — the 'centre' of the distribution.
- **Variance (σ^2):** Measures the spread/dispersion of the distribution around the mean. Standard deviation σ is its square root.
- **Why this matters for Generative AI:** Every generative model is, at its core, trying to approximate the true (unknown) data distribution p_data(x) with a model distribution p_model(x), so that sampling from p_model produces realistic data.

3.2 The Gaussian (Normal) Distribution

The Gaussian / Normal distribution is the single most important probability distribution in generative modelling — it is used as the prior/latent distribution in VAEs, the noise distribution in GANs and Diffusion Models, and appears throughout statistics due to the Central Limit Theorem.

$$f(x) = (1 / \sqrt{(2\pi\sigma^2)}) \cdot exp(-(x - \mu)^2 / (2\sigma^2))$$

Where:

- μ **(mu)** — the mean; determines the centre/location of the bell curve.
- σ^2 **(sigma squared)** — the variance; determines the width/spread of the curve (larger $\sigma^2 \rightarrow$ flatter, wider curve).
- σ — the standard deviation.
- Notation: $X \sim N(\mu, \sigma^2)$ is read as 'X follows a normal distribution with mean μ and variance σ^2.'

Key properties of the Gaussian distribution (frequently tested):

- **Symmetric, bell-shaped curve** centred exactly at the mean μ.
- **Mean = Median = Mode** for a normal distribution.
- **Empirical (68–95–99.7) Rule:** ~68% of data lies within ±1σ of the mean, ~95% within ±2σ, and ~99.7% within ±3σ.
- **Standard Normal Distribution:** the special case with $\mu = 0$ and $\sigma^2 = 1$, denoted N(0, 1); any Gaussian variable can be converted to standard form via $z = (x - \mu)/\sigma$.

- **Central Limit Theorem (CLT):** the sum/average of a large number of independent random variables tends toward a Gaussian distribution, regardless of the original distribution — this is why Gaussian noise appears so often in modelling.
- **Maximum Entropy property:** among all distributions with a given mean and variance, the Gaussian has the maximum entropy (i.e., it is the 'least assumptive'/most natural choice), which is why it is the default prior in VAEs and the default noise model in diffusion.

3.3 The Multivariate Gaussian Distribution

Real-world data (images, embeddings) is high-dimensional, so we need to extend the 1-D Gaussian to many dimensions simultaneously — this is the Multivariate Gaussian (Multivariate Normal) Distribution.

$$f(x) = 1 / ((2\pi)^{(d/2)} |\Sigma|^{(1/2)}) \cdot exp(-(1/2)(x-\mu)^{T} \Sigma^{-1} (x-\mu))$$

Where (definitions students must be able to state):

- **x** — a d-dimensional random vector (e.g., d = number of pixels or latent dimensions).
- μ **(mean vector)** — a d-dimensional vector giving the mean of each dimension.
- Σ **(covariance matrix)** — a d × d symmetric, positive semi-definite matrix; diagonal entries are the variances of each dimension, off-diagonal entries are the covariances between pairs of dimensions (i.e., how dimensions vary together).
- $|\Sigma|$ — the determinant of the covariance matrix.
- Σ^{-1} — the inverse of the covariance matrix (appears in the exponent, generalizing the $1/\sigma^2$ term).
- Notation: $X \sim N(\mu, \Sigma)$.

Important special cases and concepts:

- **Isotropic / Spherical Gaussian:** $\Sigma = \sigma^2 I$ (identity matrix scaled by a constant) — all dimensions have equal variance and are uncorrelated. This is the standard choice for the latent prior in a VAE, i.e., $z \sim N(0, I)$.
- **Diagonal Covariance Gaussian:** Σ is diagonal but entries can differ — dimensions are uncorrelated but have different variances (VAEs often predict a diagonal covariance per input).
- **Full Covariance Gaussian:** Σ has non-zero off-diagonal terms, capturing correlations between dimensions — more expressive but far more parameters to estimate (d^2 instead of d).
- **Mahalanobis Distance:** the term $(x-\mu)^{T}\Sigma^{-1}(x-\mu)$ in the exponent is the squared Mahalanobis distance — a way of measuring distance from the mean that accounts for the shape/correlation of the distribution (unlike plain Euclidean distance).
- **Geometric intuition:** contours of equal probability density of a multivariate Gaussian are ellipses (in 2D) or ellipsoids (in higher dimensions), oriented and shaped according to Σ.

3.4 Sampling Methods

DEFINITION — Sampling

The process of generating random data points (samples) x that follow a target probability distribution p(x). Sampling is the mechanism by which every generative model actually 'creates' new data once training is complete — e.g., sampling a random noise vector z and passing it through a generator network.

3.4.1 Inverse Transform Sampling

Uses the CDF F(x) of the target distribution. Steps: (1) draw u from a Uniform(0,1) distribution; (2) compute x = F■¹(u), the inverse CDF evaluated at u. The resulting x follows the target distribution. Simple and exact, but requires the inverse CDF to be known/computable in closed form, which is often not possible for complex, high-dimensional distributions.

3.4.2 Rejection Sampling

Used when the target distribution p(x) is hard to sample from directly but can be evaluated pointwise. Steps: (1) choose a simpler proposal distribution q(x) and constant M such that M·q(x) ≥ p(x) for all x (an 'envelope'); (2) sample a candidate x from q(x); (3) sample u from Uniform(0,1); (4) accept x if u ≤ p(x) / (M·q(x)), otherwise reject and repeat. Downside: can be very inefficient (high rejection rate) in high dimensions — this is one reason simple sampling methods do not scale to complex generative modelling and motivate learning neural samplers instead.

3.4.3 Markov Chain Monte Carlo (MCMC)

A family of algorithms that construct a Markov chain whose stationary (equilibrium) distribution is the target distribution p(x); after a 'burn-in' period, samples from the chain approximate samples from p(x). Useful when direct sampling is impossible and even the normalizing constant of p(x) is unknown.

- **Metropolis–Hastings Algorithm:** proposes a new state from a proposal distribution, then accepts/rejects it based on an acceptance ratio computed from p(x), ensuring the chain converges to the target distribution.
- **Gibbs Sampling:** a special case of MCMC used when we can sample from the conditional distribution of each variable given all the others; iteratively resample each dimension conditioned on the current values of the rest.
- **Langevin Dynamics / Score-based sampling:** uses the gradient of the log-probability (the 'score function' ∇■ log p(x)) to guide sampling toward high-probability regions, adding controlled noise at each step — this is the direct mathematical basis of modern **Diffusion Models** (see Section 4.3).

3.4.4 Monte Carlo Methods (general)

Broad class of techniques that use repeated random sampling to approximate numerical results — e.g., estimating an expectation E[f(x)] by averaging f(x) over many samples drawn from p(x). Monte Carlo estimation underlies training objectives such as the Evidence Lower Bound (ELBO) in VAEs, where expectations over the latent distribution are approximated using sampled latent vectors.

3.4.5 The Reparameterization Trick (bridge to VAEs)

A special sampling technique that allows gradients to flow through a random sampling step during neural network training. Instead of sampling z directly from N(μ, σ²) (which is non-differentiable), we sample a noise variable ε ~ N(0, 1) and compute z = μ + σ·ε. Now z is a deterministic, differentiable function of the learnable parameters μ and σ, so backpropagation can flow through the sampling step. This trick is essential to training Variational Autoencoders (Section 4.2).

Method	Core Idea	Typical Use Case
Inverse Transform Sampling	Apply inverse CDF to uniform random numbers	Simple 1-D known distributions
Rejection Sampling	Accept/reject candidates from an envelope proposal	Distributions known only up to evaluation

Method	Core Idea	Typical Use Case
MCMC (Metropolis-Hastings, Gibbs)	Build a Markov chain converging to target distribution	Complex, high-dimensional, unnormalized distributions
Monte Carlo Estimation	Approximate expectations via averaging many samples	Estimating losses/expectations (e.g., ELBO in VAEs)
Langevin / Score-based Sampling	Follow the score function (gradient of log p) with noise	Diffusion model sample generation
Reparameterization Trick	Rewrite random sampling as a differentiable function of noise	Backpropagation through stochastic layers (VAEs)

★ Exam Key Points — Section 3

• Gaussian PDF formula and the role of μ (location) and σ^2 (spread) — be able to write the formula from memory.

• Multivariate Gaussian replaces μ with a mean vector and σ^2 with a covariance matrix Σ; know the difference between isotropic, diagonal, and full covariance.

• Sampling methods: inverse transform (needs invertible CDF), rejection sampling (needs an envelope), MCMC/Gibbs (for complex/unnormalized distributions), Monte Carlo (for estimating expectations).

• The reparameterization trick ($z = \mu + \sigma\varepsilon$) is the key mathematical bridge connecting probability sampling to trainable neural networks — central to VAEs.

• Langevin dynamics/score-based sampling is the mathematical ancestor of Diffusion Models.

4. Types of Generative Models: GANs, VAEs, and Diffusion Models

4.1 Generative Adversarial Networks (GANs)

DEFINITION — GAN

Introduced by **Ian Goodfellow et al. in 2014**. A GAN consists of two neural networks — a **Generator (G)** and a **Discriminator (D)** — trained simultaneously in an adversarial (competitive) game, where G tries to produce fake data realistic enough to fool D, and D tries to correctly distinguish real data from G's fakes.

Architecture and components:

- **Generator (G):** Takes a random noise vector z (usually z ~ N(0, I)) as input and transforms it through a neural network (often transposed convolutions for images) into a synthetic data sample G(z).
- **Discriminator (D):** A binary classifier network that takes a sample (real from the dataset, or fake from G) and outputs the probability that the sample is real.
- **Adversarial training loop:** D is trained to maximize its ability to tell real from fake; G is trained to minimize D's ability to detect its fakes (i.e., to maximize D's error on generated samples). The two networks are trained alternately, in a minimax two-player game.

The GAN Minimax Objective (important formula):

$$\min_G \max_D V(D, G) = E[\log D(x)] + E[\log(1 - D(G(z)))]$$

Here, the first expectation is over real data x drawn from the true data distribution p_data(x), and the second is over noise z drawn from a prior p(z) (typically Gaussian). At the theoretical optimum (Nash equilibrium), the generator's distribution p_g exactly matches the real data distribution p_data, and the discriminator outputs 0.5 everywhere (i.e., it can no longer distinguish real from fake — pure guessing).

Training procedure, step by step:

- Sample a mini-batch of real data from the training set and a mini-batch of noise vectors z.
- Generate fake samples: G(z).
- Update the Discriminator's weights to better classify real vs fake (gradient ascent on D's objective).
- Update the Generator's weights to better fool the Discriminator (gradient descent on G's objective).
- Repeat for many iterations/epochs until convergence (or until training is stopped).

Known challenges of GAN training (commonly asked):

- **Mode Collapse:** the generator produces only a limited variety of outputs (or even a single output) that reliably fool the discriminator, instead of covering the full diversity of the real data distribution.
- **Training Instability:** the adversarial min-max game can oscillate rather than converge; careful balancing of G and D is required.
- **Vanishing Gradients:** if the discriminator becomes too good too quickly, the generator receives little useful gradient signal to improve.

- **Evaluation difficulty:** there is no single loss value that reliably tells you 'how good' the generated samples are; specialized metrics like Inception Score (IS) and Fréchet Inception Distance (FID) are used instead.

Notable GAN variants worth knowing by name:

- **DCGAN** (Deep Convolutional GAN) — uses convolutional layers for stable image generation.
- **Conditional GAN (cGAN)** — conditions generation on extra information such as a class label.
- **CycleGAN** — image-to-image translation without paired training examples (e.g., horse ↔ zebra).
- **StyleGAN** — produces highly realistic, controllable high-resolution faces/images with a style-based generator.
- **Pix2Pix** — paired image-to-image translation (e.g., sketches → photos).

4.2 Variational Autoencoders (VAEs)

DEFINITION — VAE

Introduced by **Kingma and Welling in 2013**. A VAE is a probabilistic extension of the classic autoencoder that learns a smooth, continuous **latent space** and can generate new data by sampling from that latent space and decoding the sample. Unlike GANs, VAEs are trained with an explicit probabilistic objective rather than an adversarial game.

Architecture and components:

- **Encoder (Recognition Network), q(z|x):** Maps an input x to parameters of a distribution over latent variables z — typically outputs a mean vector μ and a (log) variance vector σ^2 for a Gaussian, instead of a single point.
- **Latent Space, z:** A lower-dimensional probabilistic representation of the input; the prior over z is usually chosen as a standard Gaussian, z ~ N(0, I), for simplicity and tractability.
- **Sampling (Reparameterization Trick):** z is sampled as z = μ + σ ■ ε, where ε ~ N(0, I), so gradients can backpropagate through the sampling step (see Section 3.4.5).
- **Decoder (Generative Network), p(x|z):** Maps a sampled latent vector z back to the data space, reconstructing (or generating) an output resembling the original input distribution.

The VAE Loss Function — Evidence Lower Bound (ELBO):

$$L(\theta,\phi; x) = E_{q(z|x)}[\log p(x|z)] - D_{KL}(q(z|x) \blacksquare p(z))$$

The loss has two components, and both must be memorized:

- **Reconstruction Loss** — E[log p(x|z)]: measures how well the decoder can reconstruct the original input from the sampled latent z (often implemented as mean-squared error or binary cross-entropy between input and reconstruction).
- **KL Divergence Regularization Term** — D_KL(q(z|x) ■ p(z)): measures how far the learned latent distribution q(z|x) (from the encoder) is from the chosen prior p(z) = N(0, I); this term forces the latent space to stay close to a smooth, well-structured Gaussian so that new samples can be generated by simply sampling z ~ N(0, I) and decoding.
- The overall training objective is to **maximize** the ELBO (equivalently minimize the negative ELBO), which is a lower bound on the true (intractable) data log-likelihood log p(x).

Why the latent space matters — generation process at test time:

- Once trained, new data can be generated WITHOUT using the encoder at all: simply sample a random z ~ N(0, I) and pass it through the trained Decoder to obtain a new synthetic sample.
- Because the KL term keeps q(z|x) close to a smooth Gaussian prior, the latent space is continuous and interpolatable — moving smoothly between two latent points produces a smooth morph between the corresponding outputs (e.g., morphing one face into another).

Known limitations of VAEs:

- Generated samples tend to be **blurrier** / less sharp than GAN outputs, because the pixel-wise reconstruction loss (e.g., MSE) encourages averaging over plausible outputs.
- There is a fundamental trade-off between reconstruction accuracy and how regularized/smooth the latent space is (controlled by the balance between the two ELBO terms).

4.3 Diffusion Models

DEFINITION — Diffusion Model

A class of generative models (e.g., **Denoising Diffusion Probabilistic Models, DDPM**, 2020) that learn to generate data by reversing a gradual noising process. Data is progressively destroyed by adding small amounts of Gaussian noise over many steps (the **forward process**), and a neural network is trained to reverse this process step by step (the **reverse process**), eventually turning pure random noise back into a realistic data sample.

4.3.1 The Forward (Diffusion) Process

Starting from a real data sample x■, Gaussian noise is added gradually over T time steps (T can be several hundred to a few thousand), producing a sequence of increasingly noisy versions x■, x■, …, x_T, until x_T is approximately pure Gaussian noise, x_T ~ N(0, I). Each forward step is defined as a simple, fixed (non-learned) Gaussian transition:

$$q(x_t \mid x_{t-1}) = N(x_t ; \sqrt{(1 - \beta_t)} \cdot x_{t-1} , \beta_t \cdot I)$$

Here β_t is a small variance ('noise schedule') that controls how much noise is added at step t. A convenient property is that x_t can be sampled directly from x■ in one shot without simulating all intermediate steps, using a cumulative product of the schedule.

4.3.2 The Reverse (Denoising) Process

The reverse process learns to undo the noising, step by step, starting from pure noise x_T and gradually predicting a slightly less noisy x_{T−1}, x_{T−2}, ..., down to a clean sample x■. Since the true reverse conditional is intractable, a neural network (commonly a **U-Net** architecture) is trained to approximate it:

$$p_\theta(x_{t-1} \mid x_t) = N(x_{t-1} ; \mu_\theta(x_t, t) , \Sigma_\theta(x_t, t))$$

In practice, DDPM simplifies training by having the network predict the **noise** ε that was added at each step, rather than predicting the mean directly. The training loss then becomes a simple mean-squared error between the true noise and the predicted noise:

$$L_{simple} = E[\blacksquare \varepsilon - \varepsilon_\theta(x_t, t) \blacksquare^2]$$

4.3.3 Generation (Sampling) at Test Time

- Start with a sample of pure random Gaussian noise, x_T ~ N(0, I).

- Iteratively apply the trained denoising network to predict and subtract out noise, moving from x_T → x_{T−1} → … → x■, over many steps.
- The final x■ is a newly generated, realistic data sample (e.g., an image).
- This iterative process is why diffusion models are typically slower to sample from than GANs/VAEs (which generate in a single forward pass), though faster samplers (e.g., DDIM) have been developed to reduce the number of steps needed.

4.3.4 Connection to Score-Based Generative Models

Diffusion models are closely related to score-based generative modelling, where the network learns the **score function** — the gradient of the log probability density, ∇■ log p(x) — at various noise levels, and generation is performed by following this score using Langevin-dynamics-style sampling (introduced in Section 3.4.3). This theoretical connection unifies DDPMs and score-based models under a single stochastic differential equation (SDE) framework.

Advantages and limitations of Diffusion Models:

- **Advantages:** very high sample quality/diversity (state of the art for image synthesis, e.g., Stable Diffusion, DALL·E 2/3, Midjourney), stable training (no adversarial min-max game, so no mode collapse like GANs).
- **Limitations:** slow sampling (many sequential denoising steps required), computationally expensive to train and to sample from compared to GANs and VAEs.

4.4 Comparative Summary — GANs vs VAEs vs Diffusion Models

Aspect	GAN	VAE	Diffusion Model
Core mechanism	Adversarial game between Generator and Discriminator	Encoder–Decoder with probabilistic latent space, trained via ELBO	Iterative denoising: reverse a gradual Gaussian noising process
Training objective	Minimax loss (game-theoretic)	Maximize ELBO (reconstruction – KL divergence)	Minimize noise-prediction error (MSE) at each step
Sample quality	Sharp, high-fidelity, but risk of mode collapse	Tends to be blurrier / less sharp	State-of-the-art quality and diversity
Sampling speed	Fast (single forward pass)	Fast (single forward pass)	Slow (many sequential steps, though accelerated samplers exist)
Training stability	Can be unstable (adversarial dynamics)	Stable (well-defined probabilistic objective)	Stable (no adversarial component)
Latent space	Implicit, less structured	Explicit, smooth, continuous, interpolatable	Implicit via noise trajectory
Example models	DCGAN, StyleGAN, CycleGAN, Pix2Pix	Vanilla VAE, Conditional VAE, β-VAE	DDPM, DDIM, Stable Diffusion, Imagen

★ Exam Key Points — Section 4

• GAN = Generator + Discriminator playing a minimax adversarial game; risk: mode collapse & instability.

• VAE = Encoder + Decoder trained via ELBO = Reconstruction Loss – KL Divergence; uses the reparameterization trick; produces a smooth latent space but blurrier outputs.

• Diffusion Model = forward noising process + learned reverse denoising process (usually a U-Net); state-of-the-art quality, but slow, iterative sampling.

• Know the year/author each was introduced: GAN — Goodfellow, 2014; VAE — Kingma & Welling, 2013; DDPM — Ho et al., 2020.

• Be ready to compare all three across: training objective, sample quality, sampling speed, and stability — this is a very common table-style exam question.

5. Key Applications of Generative AI

5.1 Image Synthesis

- **Text-to-image generation:** creating photorealistic or artistic images from natural-language text prompts (e.g., DALL·E, Stable Diffusion, Midjourney).
- **Image super-resolution:** enhancing low-resolution images into high-resolution versions using generative upsampling networks.
- **Image inpainting/outpainting:** filling in missing or damaged regions of an image, or extending an image beyond its original borders, in a visually consistent way.
- **Style transfer:** applying the artistic style of one image (e.g., a painting) to the content of another image.
- **Face generation and editing:** generating entirely new, non-existent human faces (e.g., StyleGAN's 'this person does not exist') and editing facial attributes (age, expression, hairstyle).
- **Data augmentation for computer vision:** generating synthetic training images to expand limited datasets, improving downstream classifier robustness.
- **Medical imaging:** synthesizing MRI/CT scan variations to help train diagnostic models when real labelled medical data is scarce.

5.2 Text Generation

- **Large Language Models (LLMs):** Transformer-based autoregressive generative models (e.g., GPT family) that generate coherent, contextually relevant text token by token.
- **Conversational AI / chatbots:** customer support agents, virtual assistants, and general-purpose assistants built on generative text models.
- **Content creation:** automated writing of articles, marketing copy, product descriptions, poetry, and creative fiction.
- **Summarization:** condensing long documents into concise summaries while preserving key information.
- **Machine translation:** generating fluent text in a target language conditioned on source-language input.
- **Code generation:** producing functional program code from natural-language descriptions (e.g., coding assistants).
- **Question answering:** generating direct, contextually grounded answers to user queries rather than just retrieving documents.

5.3 Drug Discovery

Generative AI is transforming pharmaceutical research by generating and evaluating candidate molecules computationally, dramatically speeding up early-stage discovery compared to purely experimental, trial-and-error approaches.

- **De novo molecule generation:** generative models (GANs, VAEs, diffusion models adapted to molecular graphs/SMILES strings) propose entirely new candidate molecules with desired chemical/biological properties.
- **Protein structure prediction and design:** generating novel protein sequences/structures with specific functions (related tools: AlphaFold-style structure prediction combined with generative design).

- **Property optimization:** generating molecules optimized for target properties such as binding affinity to a disease target, solubility, and low toxicity, often guided by reinforcement learning on top of a generative model.
- **Virtual screening acceleration:** generating and filtering large libraries of plausible drug-like compounds computationally before committing to expensive lab synthesis and testing.
- **Reduced cost and time:** traditional drug discovery can take over a decade and cost billions of dollars; generative approaches aim to shorten early discovery phases significantly by narrowing the search space intelligently.

5.4 Other Notable Applications

- **Audio and music generation:** composing original music, synthesizing realistic speech (text-to-speech), and voice cloning/conversion.
- **Video generation:** synthesizing new video clips from text prompts, frame interpolation, and video prediction.
- **3D content and game asset generation:** generating 3D models, textures, and environments for gaming, animation, and AR/VR.
- **Synthetic tabular data:** generating privacy-preserving synthetic datasets that statistically resemble sensitive real data (useful in finance, healthcare) without exposing actual records.
- **Anomaly detection:** using a generative model's learned notion of 'normal' data distribution to flag inputs that are poorly reconstructed/explained as anomalies (e.g., fraud detection, industrial defect detection).
- **Fashion and industrial design:** generating new clothing designs, product concepts, and CAD prototypes.
- **Data compression:** latent representations learned by generative models (especially VAEs) can be used as compact, information-dense codes for compression.

★ Exam Key Points — Section 5

• Be able to name at least 2 concrete applications each for Image Synthesis, Text Generation, and Drug Discovery — this is the most commonly asked application-based question.

• Drug discovery example flow: generate candidate molecule → predict/optimize properties → virtual screening → shortlist for lab testing.

• Generative AI applications span far beyond images/text: audio, video, 3D, tabular/synthetic data, and anomaly detection are all valid and testable examples.

Unit 1 — Quick Revision Summary

Use this page the night before the exam to rapidly recall every sub-topic in Unit 1.

1. AI, ML, DL

- AI ⊃ ML ⊃ DL (nested subsets).
- AI types: Narrow / General / Super; Reactive / Limited Memory / Theory of Mind / Self-Aware.
- ML types: Supervised, Unsupervised, Semi-supervised, Reinforcement.
- DL = multi-layer neural networks; rose due to Big Data + Compute (GPU) + Algorithms (2012 AlexNet).

2. Generative Models

- Generative models learn P(x) or P(x,y); discriminative models learn P(y|x).
- Only generative models can synthesize new samples.
- Scope: images, text, audio, video, 3D, molecules/proteins, tabular data.

3. Probability Foundations

- Gaussian PDF: $f(x) = (1/\sqrt{(2\pi\sigma^2)}) \cdot \exp(-(x-\mu)^2/2\sigma^2)$.
- Multivariate Gaussian uses mean vector μ and covariance matrix Σ; isotropic vs diagonal vs full covariance.
- Sampling: Inverse Transform, Rejection Sampling, MCMC (Metropolis-Hastings, Gibbs), Monte Carlo, Langevin/score-based, Reparameterization trick ($z = \mu + \sigma\varepsilon$).

4. GANs vs VAEs vs Diffusion

- GAN: Generator + Discriminator, minimax loss, sharp images, risk of mode collapse. (Goodfellow, 2014)
- VAE: Encoder + Decoder, ELBO = Reconstruction – KL Divergence, smooth latent space, blurrier output. (Kingma & Welling, 2013)
- Diffusion: forward noising + learned reverse denoising (U-Net), state-of-the-art quality, slow sampling. (Ho et al., DDPM, 2020)

5. Applications

- Image Synthesis: text-to-image, super-resolution, inpainting, style transfer, face generation.
- Text Generation: LLMs/chatbots, summarization, translation, code generation.
- Drug Discovery: de novo molecule generation, protein design, property optimization, virtual screening.
- Others: audio/music, video, 3D assets, synthetic tabular data, anomaly detection.

End of Unit 1 Detailed Notes.